

Projet B - Séquence 4.8 -

Script de création de la base de données (2)

Voici le **Modèle Conceptuel de Données (MCD)** mis à jour pour **StudyFlow AI**, intégrant **toutes les entités, champs et relations** identifiés dans vos scénarios, tout en conservant la structure existante de votre script SQL (tables comme `Question_Quiz`, `Réponse_Quiz`, `Progression`, etc.). J'ai **ajouté les compléments** des scénarios (ex: `Demande_Reset_MotDePasse`, `Salle`, `Participation_Séance`) et **harmonisé les conventions** (ex: `TIMESTAMP WITH TIME ZONE`, `DEFAULT NOW()`).

MCD Complet pour StudyFlow AI (Version Simulée)

◆ Légende

- **Entités** : Tables SQL (ex: `Utilisateur`, `Groupe`).
- **Relations** : Clés étrangères (ex: `groupe_id UUID REFERENCES Groupe(id)`).
- **Champs** : Colonnes avec types et contraintes.
-  **Nouveautés** : Entités/champs ajoutés pour les scénarios.
-  **Modifications** : Champs existants ajustés (ex: ajout de `statut_compte` dans `Utilisateur`).

Script SQL Complet (Mis à Jour)

Voici le **script SQL harmonisé** avec :

1. **Toutes vos tables existent** (`Question_Quiz`, `Réponse_Quiz`, etc.).
2. **Les ajouts des scénarios** (`Demande_Reset_MotDePasse`, `Salle`, etc.).
3. **Les conventions unifiées** (`TIMESTAMP WITH TIME ZONE`, `DEFAULT NOW()`).

```
-- =====
-- 1. EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

---
```

```

-- =====
-- 2. ÉNUMÉRATIONS
-- =====
-- Statut utilisateur
CREATE TYPE statut_utilisateur AS ENUM ('actif', 'bloqué', 'suspendu', 'non_verifié');

-- Statut demande de reset
CREATE TYPE statut_demande_reset AS ENUM ('en_attente', 'utilisé', 'expiré');

-- Statut séance
CREATE TYPE statut_séance AS ENUM ('planifiée', 'confirmée', 'annulée');

-- Statut participation
CREATE TYPE statut_participation AS ENUM ('validé', 'refusé', 'en_attente');

-- Statut tâche
CREATE TYPE statut_tâche AS ENUM ('à_faire', 'en_cours', 'terminée');

-- Priorité tâche
CREATE TYPE priorité_tâche AS ENUM ('basse', 'moyenne', 'haute');

-- Type ressource
CREATE TYPE type_ressource AS ENUM ('fichier', 'lien', 'texte');

-- Type événement
CREATE TYPE type_événement AS ENUM ('tâche', 'séance', 'appel', 'échéance');

-- Type notification
CREATE TYPE type_notification AS ENUM (
    'réservation_salle',
    'invitation_groupe',
    'invitation_séance',
    'rappel_séance',
    'nouvelle_tâche',
    'nouvelle_ressource',
    'commentaire_note',
    'réinitialisation_mot_de_passe'
);

-- Rôle dans un groupe
CREATE TYPE rôle_groupe AS ENUM ('membre', 'admin', 'créateur');

-- Statut salle
CREATE TYPE statut_salle AS ENUM ('disponible', 'occupée', 'maintenance');

-- Type question quiz (existant)
CREATE TYPE type_question_quiz AS ENUM ('QCM', 'QCU', 'texte_libre', 'vrai_faux');

-- Type données analytiques (existant)
CREATE TYPE type_donnees_analytiques AS ENUM ('temps_étude', 'notes_moyennes',
'tâches_complétées');

-- Statut cours (existant)
CREATE TYPE statut_cours AS ENUM ('actif', 'archivé', 'en_création');

```

-- =====

-- 3. TABLES NOUVELLES (SCÉNARIOS)

-- =====

-- Table Utilisateur (modifiée)

```
CREATE TABLE utilisateur (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  nom VARCHAR(100) NOT NULL,  
  prénom VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  mot_de_passe TEXT NOT NULL,  
  rôle VARCHAR(20) NOT NULL CHECK (rôle IN ('étudiant', 'professeur', 'gestionnaire')),  
  statut_statut_utilisateur NOT NULL DEFAULT 'non_vérifié',  
  dernière_connexion TIMESTAMP WITH TIME ZONE,  
  dernière_tentative_connexion TIMESTAMP WITH TIME ZONE,  
  compteur_echecs_connexion INTEGER NOT NULL DEFAULT 0,  
  date_inscription TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
  email_vérifié BOOLEAN NOT NULL DEFAULT FALSE,  
  jeton_verification_email UUID,  
  date_expiration_jeton_verification TIMESTAMP WITH TIME ZONE,  
  photo_profil TEXT,  
  CONSTRAINT valid_email CHECK (email ~* '^[A-Za-z0-9._%~]+@[A-Za-z0-9.-]+.[A-Za-z]+$')  
);
```

-- Table Demande_Reset_MotDePasse

```
CREATE TABLE demande_reset_mot_de_passe (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  utilisateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,  
  jeton_reinitialisation UUID NOT NULL UNIQUE DEFAULT uuid_generate_v4(),  
  date_expiration_jeton TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW() + INTERVAL '1  
hour',  
  statut_statut_demande_reset NOT NULL DEFAULT 'en_attente',  
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()  
);
```

-- Table Groupe (renommée de groupe_collaboratif pour cohérence)

```
CREATE TABLE groupe (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  nom VARCHAR(255) NOT NULL,  
  description TEXT,  
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
  créateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,  
  statut VARCHAR(20) NOT NULL DEFAULT 'actif' CHECK (statut IN ('actif', 'archivé')),  
  image_groupe TEXT  
);
```

-- Table Utilisateur_Groupe (remplace membre_groupe)

```
CREATE TABLE utilisateur_groupe (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  groupe_id UUID NOT NULL REFERENCES groupe(id) ON DELETE CASCADE,  
  utilisateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
```

```

rôle rôle_groupe NOT NULL DEFAULT 'membre',
date_adhésion TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
UNIQUE(groupe_id, utilisateur_id)
);

-- Table Salle
CREATE TABLE salle (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(100) NOT NULL,
  description TEXT,
  capacité INTEGER NOT NULL DEFAULT 10,
  statut statut_salle NOT NULL DEFAULT 'disponible',
  localisation VARCHAR(255),
  équipement TEXT[]
);

-- Table Réservation_Salle
CREATE TABLE réservation_salle (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  salle_id UUID NOT NULL REFERENCES salle(id) ON DELETE CASCADE,
  date_debut TIMESTAMP WITH TIME ZONE NOT NULL,
  date_fin TIMESTAMP WITH TIME ZONE NOT NULL,
  statut VARCHAR(20) NOT NULL DEFAULT 'confirmée' CHECK (statut IN ('confirmée', 'annulée')),
  groupe_id UUID REFERENCES groupe(id) ON DELETE SET NULL,
  utilisateur_id UUID REFERENCES utilisateur(id) ON DELETE SET NULL,
  CONSTRAINT valid_dates CHECK (date_fin > date_debut)
);

-- Table Séance
CREATE TABLE séance (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  groupe_id UUID NOT NULL REFERENCES groupe(id) ON DELETE CASCADE,
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  date_debut TIMESTAMP WITH TIME ZONE NOT NULL,
  date_fin TIMESTAMP WITH TIME ZONE NOT NULL,
  durée INTEGER NOT NULL, -- En minutes
  lieu VARCHAR(255),
  statut statut_séance NOT NULL DEFAULT 'planifiée',
  obligatoire BOOLEAN NOT NULL DEFAULT FALSE,
  créateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  CONSTRAINT valid_séance_dates CHECK (date_fin > date_debut)
);

-- Table Participation_Séance
CREATE TABLE participation_séance (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  séance_id UUID NOT NULL REFERENCES séance(id) ON DELETE CASCADE,
  utilisateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  statut statut_participation NOT NULL DEFAULT 'en_attente',
  date_validation TIMESTAMP WITH TIME ZONE,
  UNIQUE(séance_id, utilisateur_id)
);

```

```

-- Table Note
CREATE TABLE note (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  groupe_id UUID NOT NULL REFERENCES groupe(id) ON DELETE CASCADE,
  auteur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  contenu TEXT NOT NULL,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  date_modification TIMESTAMP WITH TIME ZONE,
  titre VARCHAR(255)
);

-- Table Commentaire_Note
CREATE TABLE commentaire_note (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  note_id UUID NOT NULL REFERENCES note(id) ON DELETE CASCADE,
  auteur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  contenu TEXT NOT NULL,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Table Notification (modifiée)
CREATE TABLE notification (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  destinataire_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  type type_notification NOT NULL,
  contenu TEXT NOT NULL,
  lien TEXT,
  date_envoi TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  statut VARCHAR(20) NOT NULL DEFAULT 'non_lu' CHECK (statut IN ('non_lu', 'lu')),
  expéditeur_id UUID REFERENCES utilisateur(id) ON DELETE SET NULL
);

---
-- =====
-- 4. TABLES EXISTANTES (DE VOTRE SCRIPT)
-- =====

-- Table Établissement
CREATE TABLE etablissement (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(100) NOT NULL,
  description TEXT
);

-- Table Cursus
CREATE TABLE cursus (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(100) NOT NULL,
  description TEXT,
  id_établissement UUID REFERENCES etablissement(id) ON DELETE CASCADE
);

-- Table Cours

```

```

CREATE TABLE cours (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  id_professeur UUID REFERENCES utilisateur(id) ON DELETE CASCADE,
  id_cursus UUID REFERENCES cursus(id) ON DELETE CASCADE,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  matière VARCHAR(100),
  statut statut_cours NOT NULL DEFAULT 'actif'
);

-- Table Utilisateur_Cours
CREATE TABLE utilisateur_cours (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  utilisateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  cours_id UUID NOT NULL REFERENCES cours(id) ON DELETE CASCADE,
  date_inscription TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  UNIQUE(utilisateur_id, cours_id)
);

-- Table Calendrier
CREATE TABLE calendrier (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  nom VARCHAR(100) NOT NULL,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Table Tâche (modifiée pour inclure calendrier_id)
CREATE TABLE tâche (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  groupe_id UUID REFERENCES groupe(id) ON DELETE CASCADE,
  utilisateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  date_limite TIMESTAMP WITH TIME ZONE,
  statut statut_tâche NOT NULL DEFAULT 'à_faire',
  priorité priorité_tâche NOT NULL DEFAULT 'moyenne',
  date_modification TIMESTAMP WITH TIME ZONE,
  calendrier_id UUID REFERENCES calendrier(id) ON DELETE SET NULL
);

-- Table Échéance
CREATE TABLE échéance (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_tâche UUID NOT NULL REFERENCES tâche(id) ON DELETE CASCADE,
  date_limite TIMESTAMP WITH TIME ZONE NOT NULL,
  description TEXT
);

-- Table Événement (modifiée pour inclure tous les types)
CREATE TABLE événement (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```

```

type type_événement NOT NULL,
titre VARCHAR(255) NOT NULL,
description TEXT,
date_debut TIMESTAMP WITH TIME ZONE NOT NULL,
date_fin TIMESTAMP WITH TIME ZONE NOT NULL,
tâche_id UUID REFERENCES tâche(id) ON DELETE CASCADE,
séance_id UUID REFERENCES séance(id) ON DELETE CASCADE,
groupe_id UUID REFERENCES groupe(id) ON DELETE CASCADE,
utilisateur_id UUID REFERENCES utilisateur(id) ON DELETE CASCADE,
statut VARCHAR(20) NOT NULL DEFAULT 'actif' CHECK (statut IN ('actif', 'annulé')),
calendrier_id UUID REFERENCES calendrier(id) ON DELETE SET NULL,
couleur VARCHAR(7) DEFAULT '#3498db'
);

-- Table Ressource (modifiée pour inclure groupe_id)
CREATE TABLE ressource (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  groupe_id UUID REFERENCES groupe(id) ON DELETE CASCADE,
  utilisateur_id UUID REFERENCES utilisateur(id) ON DELETE CASCADE,
  id_cours UUID REFERENCES cours(id) ON DELETE CASCADE,
  titre VARCHAR(255) NOT NULL,
  type type_ressource NOT NULL,
  url TEXT NOT NULL,
  description TEXT,
  date_ajout TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  note_moyenne FLOAT DEFAULT 0
);

-- Table Note_Ressource
CREATE TABLE note_ressource (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_ressource UUID NOT NULL REFERENCES ressource(id) ON DELETE CASCADE,
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  note FLOAT NOT NULL CHECK (note BETWEEN 0 AND 5),
  date_notation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  UNIQUE(id_ressource, id_utilisateur)
);

-- Table Quiz
CREATE TABLE quiz (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_cours UUID REFERENCES cours(id) ON DELETE CASCADE,
  id_utilisateur UUID REFERENCES utilisateur(id) ON DELETE CASCADE,
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Table Question_Quiz
CREATE TABLE question_quiz (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  id_quiz UUID NOT NULL REFERENCES quiz(id) ON DELETE CASCADE,
  contenu TEXT NOT NULL,
  type type_question_quiz NOT NULL,

```

```

    points INT NOT NULL DEFAULT 1
);

-- Table Réponse_Quiz
CREATE TABLE reponse_quiz (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_question_quiz UUID NOT NULL REFERENCES question_quiz(id) ON DELETE CASCADE,
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    contenu TEXT,
    est_correcte BOOLEAN NOT NULL DEFAULT FALSE,
    date_soumission TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    UNIQUE (id_question_quiz, id_utilisateur)
);

-- Table Progression
CREATE TABLE progression (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    id_tâche UUID REFERENCES tâche(id) ON DELETE CASCADE,
    id_quiz UUID REFERENCES quiz(id) ON DELETE CASCADE,
    note FLOAT CHECK (note BETWEEN 0 AND 20),
    date_realisation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Table Données_Analytiques
CREATE TABLE donnees_analytiques (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    type type_donnees_analytiques NOT NULL,
    valeur FLOAT NOT NULL,
    date_enregistrement TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

---
-- =====
-- 5. INDEX POUR L'OPTIMISATION
-- =====
-- Index pour Utilisateur
CREATE INDEX idx_utilisateur_email ON utilisateur(email);
CREATE INDEX idx_utilisateur_statut ON utilisateur(statut);
CREATE INDEX idx_utilisateur_derniere_connexion ON utilisateur(derniere_connexion);

-- Index pour Demande_Reset_MotDePasse
CREATE INDEX idx_demande_reset_jeton ON demande_reset_mot_de_passe(jeton_reinitialisation);
CREATE INDEX idx_demande_reset_utilisateur ON demande_reset_mot_de_passe(utilisateur_id);

-- Index pour Groupe
CREATE INDEX idx_groupe_créateur ON groupe(créateur_id);
CREATE INDEX idx_groupe_nom ON groupe(nom);

-- Index pour Utilisateur_Groupe
CREATE INDEX idx_utilisateur_groupe_groupe ON utilisateur_groupe(groupe_id);
CREATE INDEX idx_utilisateur_groupe_utilisateur ON utilisateur_groupe(utilisateur_id);

```



```
-- Index pour Salle
CREATE INDEX idx_salle_nom ON salle(nom);

-- Index pour Réservation_Salle
CREATE INDEX idx_reservation_salle_salle_dates ON reservation_salle(salle_id, date_debut, date_fin);
CREATE INDEX idx_reservation_salle_groupe ON reservation_salle(groupe_id);

-- Index pour Séance
CREATE INDEX idx_séance_groupe ON séance(groupe_id);
CREATE INDEX idx_séance_dates ON séance(date_debut, date_fin);
CREATE INDEX idx_séance_créateur ON séance(créateur_id);

-- Index pour Participation_Séance
CREATE INDEX idx_participation_séance_séance ON participation_séance(séance_id);
CREATE INDEX idx_participation_séance_utilisateur ON participation_séance(utilisateur_id);

-- Index pour Note
CREATE INDEX idx_note_groupe ON note(groupe_id);
CREATE INDEX idx_note_auteur ON note(auteur_id);

-- Index pour Commentaire_Note
CREATE INDEX idx_commentaire_note ON commentaire_note(note_id);
CREATE INDEX idx_commentaire_auteur ON commentaire_note(auteur_id);

-- Index pour Notification
CREATE INDEX idx_notification_destinataire ON notification(destinataire_id);
CREATE INDEX idx_notification_type ON notification(type);
CREATE INDEX idx_notification_statut ON notification(statut);

-- Index pour Tâche
CREATE INDEX idx_tâche_utilisateur ON tâche(utilisateur_id);
CREATE INDEX idx_tâche_groupe ON tâche(groupe_id);
CREATE INDEX idx_tâche_statut ON tâche(statut);
CREATE INDEX idx_tâche_priorite ON tâche(priorité);
CREATE INDEX idx_tâche_date_limite ON tâche(date_limite);
CREATE INDEX idx_tâche_calendrier ON tâche(calendrier_id);

-- Index pour Échéance
CREATE INDEX idx_échéance_tâche ON échéance(id_tâche);
CREATE INDEX idx_échéance_date_limite ON échéance(date_limite);

-- Index pour Événement
CREATE INDEX idx_événement_type ON événement(type);
CREATE INDEX idx_événement_dates ON événement(date_debut, date_fin);
CREATE INDEX idx_événement_groupe ON événement(groupe_id);
CREATE INDEX idx_événement_utilisateur ON événement(utilisateur_id);
CREATE INDEX idx_événement_calendrier ON événement(calendrier_id);

-- Index pour Ressource
CREATE INDEX idx_ressource_groupe ON ressource(groupe_id);
CREATE INDEX idx_ressource_utilisateur ON ressource(utilisateur_id);
CREATE INDEX idx_ressource_cours ON ressource(id_cours);
CREATE INDEX idx_ressource_type ON ressource(type);
```

```

-- Index pour Note_Ressource
CREATE INDEX idx_note_ressource_ressource ON note_ressource(id_ressource);
CREATE INDEX idx_note_ressource_utilisateur ON note_ressource(id_utilisateur);

-- Index pour Quiz
CREATE INDEX idx_quiz_cours ON quiz(id_cours);
CREATE INDEX idx_quiz_utilisateur ON quiz(id_utilisateur);

-- Index pour Question_Quiz
CREATE INDEX idx_question_quiz_quiz ON question_quiz(id_quiz);

-- Index pour Réponse_Quiz
CREATE INDEX idx_reponse_quiz_question ON reponse_quiz(id_question_quiz);
CREATE INDEX idx_reponse_quiz_utilisateur ON reponse_quiz(id_utilisateur);

-- Index pour Progression
CREATE INDEX idx_progression_utilisateur ON progression(id_utilisateur);
CREATE INDEX idx_progression_tâche ON progression(id_tâche);
CREATE INDEX idx_progression_quiz ON progression(id_quiz);

-- Index pour Données_Analytiques
CREATE INDEX idx_donnees_analytiques_utilisateur ON donnees_analytiques(id_utilisateur);
CREATE INDEX idx_donnees_analytiques_type ON donnees_analytiques(type);

---
-- =====
-- 6. TRIGGERS
-- =====

-- Trigger pour mettre à jour dernière_connexion
CREATE OR REPLACE FUNCTION update_derniere_connexion()
RETURNS TRIGGER AS $$
BEGIN
    NEW.dernière_connexion = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_update_derniere_connexion
BEFORE UPDATE ON utilisateur
FOR EACH ROW
EXECUTE FUNCTION update_derniere_connexion();

---

-- Trigger pour bloquer un compte après 3 échecs de connexion
CREATE OR REPLACE FUNCTION check_login_failures()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.compteur_echecs_connexion >= 3 THEN
        NEW.statut = 'bloqué';
    END IF;
    RETURN NEW;
END;

```

```

$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_block_account_after_failures
BEFORE UPDATE ON utilisateur
FOR EACH ROW
WHEN (OLD.compteur_echecs_connexion < 3 AND NEW.compteur_echecs_connexion >= 3)
EXECUTE FUNCTION check_login_failures();

---
-- Trigger pour synchroniser Tâche → Événement
CREATE OR REPLACE FUNCTION sync_tâche_événement()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
        -- Créer ou mettre à jour un événement pour la tâche
        INSERT INTO événement (
            type, titre, description, date_debut, date_fin,
            calendrier_id, couleur, tâche_id, utilisateur_id
        )
        VALUES (
            'tâche',
            NEW.titre,
            NEW.description,
            NEW.date_limite,
            NEW.date_limite + INTERVAL '1 hour',
            NEW.calendrier_id,
            '#FF5733',
            NEW.id,
            NEW.utilisateur_id
        )
        ON CONFLICT (tâche_id)
        DO UPDATE SET
            titre = EXCLUDED.titre,
            description = EXCLUDED.description,
            date_debut = EXCLUDED.date_debut,
            date_fin = EXCLUDED.date_fin,
            calendrier_id = EXCLUDED.calendrier_id;

        -- Synchroniser les échéances de la tâche
        INSERT INTO événement (
            type, titre, description, date_debut, date_fin,
            calendrier_id, couleur, échéance_id
        )
        SELECT
            'échéance',
            e.description,
            e.description,
            e.date_limite,
            e.date_limite + INTERVAL '1 hour',
            NEW.calendrier_id,
            '#33FF57',
            e.id
        FROM échéance e
        WHERE e.id_tâche = NEW.id
    
```

```

ON CONFLICT (échéance_id)
DO UPDATE SET
    titre = EXCLUDED.titre,
    description = EXCLUDED.description,
    date_debut = EXCLUDED.date_debut,
    date_fin = EXCLUDED.date_fin,
    calendrier_id = EXCLUDED.calendrier_id;

ELSIF TG_OP = 'DELETE' THEN
    -- Supprimer les événements liés à la tâche
    DELETE FROM événement WHERE tâche_id = OLD.id;
    -- Supprimer les événements liés aux échéances de la tâche
    DELETE FROM événement WHERE échéance_id IN (
        SELECT id FROM échéance WHERE id_tâche = OLD.id
    );
END IF;
RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_sync_tâche_événement
AFTER INSERT OR UPDATE OR DELETE ON tâche
FOR EACH ROW
EXECUTE FUNCTION sync_tâche_événement();

---
-- Trigger pour synchroniser Échéance → Événement
CREATE OR REPLACE FUNCTION sync_échéance_événement()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
        INSERT INTO événement (
            type, titre, description, date_debut, date_fin,
            calendrier_id, couleur, échéance_id
        )
        VALUES (
            'échéance',
            NEW.description,
            NEW.description,
            NEW.date_limite,
            NEW.date_limite + INTERVAL '1 hour',
            (SELECT calendrier_id FROM tâche WHERE id = NEW.id_tâche),
            '#33FF57',
            NEW.id
        )
    )
    ON CONFLICT (échéance_id)
    DO UPDATE SET
        titre = EXCLUDED.titre,
        description = EXCLUDED.description,
        date_debut = EXCLUDED.date_debut,
        date_fin = EXCLUDED.date_fin,
        calendrier_id = EXCLUDED.calendrier_id;

ELSIF TG_OP = 'DELETE' THEN

```

```

        DELETE FROM événement WHERE échéance_id = OLD.id;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trigger_sync_échéance_événement
AFTER INSERT OR UPDATE OR DELETE ON échéance
FOR EACH ROW
EXECUTE FUNCTION sync_échéance_événement();

```

```

-- Trigger pour synchroniser Séance → Événement
CREATE OR REPLACE FUNCTION sync_séance_événement()
RETURNS TRIGGER AS $$

```

```

DECLARE

```

```

    membre RECORD;

```

```

BEGIN

```

```

    IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN

```

```

        -- Créer ou mettre à jour un événement pour la séance

```

```

        INSERT INTO événement (
            type, titre, description, date_debut, date_fin,
            séance_id, groupe_id, couleur
        )

```

```

        VALUES (
            'séance',
            NEW.titre,
            NEW.description,
            NEW.date_debut,
            NEW.date_fin,
            NEW.id,
            NEW.groupe_id,
            '#9b59b6'
        )

```

```

    )
    ON CONFLICT (séance_id)
    DO UPDATE SET

```

```

        titre = EXCLUDED.titre,
        description = EXCLUDED.description,
        date_debut = EXCLUDED.date_debut,
        date_fin = EXCLUDED.date_fin,
        groupe_id = EXCLUDED.groupe_id;

```

```

-- Créer des événements pour tous les membres du groupe

```

```

FOR membre IN

```

```

    SELECT ug.utilisateur_id
    FROM utilisateur_groupe ug
    WHERE ug.groupe_id = NEW.groupe_id

```

```

LOOP

```

```

    INSERT INTO événement (
        type, titre, description, date_debut, date_fin,
        séance_id, utilisateur_id, groupe_id, couleur
    )

```

```

    VALUES (
        'séance',

```

```

        NEW.titre,
        NEW.description,
        NEW.date_debut,
        NEW.date_fin,
        NEW.id,
        membre.utilisateur_id,
        NEW.groupe_id,
        '#9b59b6'
    )
    ON CONFLICT (séance_id, utilisateur_id)
    DO UPDATE SET
        titre = EXCLUDED.titre,
        description = EXCLUDED.description,
        date_debut = EXCLUDED.date_debut,
        date_fin = EXCLUDED.date_fin;
END LOOP;

ELSIF TG_OP = 'DELETE' THEN
    DELETE FROM événement WHERE séance_id = OLD.id;
END IF;
RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_sync_séance_événement
AFTER INSERT OR UPDATE OR DELETE ON séance
FOR EACH ROW
EXECUTE FUNCTION sync_séance_événement();

---
-- Trigger pour protéger le créateur du groupe
CREATE OR REPLACE FUNCTION protect_créateur_groupe()
RETURNS TRIGGER AS $$
DECLARE
    est_créateur BOOLEAN;
BEGIN
    IF TG_OP = 'DELETE' THEN
        SELECT groupe.créateur_id = OLD.utilisateur_id INTO est_créateur
        FROM groupe
        WHERE groupe.id = OLD.groupe_id;

        IF est_créateur THEN
            RAISE EXCEPTION 'Impossible de supprimer le créateur du groupe.';
        END IF;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_protect_créateur_groupe
BEFORE DELETE ON utilisateur_groupe
FOR EACH ROW
EXECUTE FUNCTION protect_créateur_groupe();

```

```

---
-- Trigger pour recalculer note_moyenne dans Ressource
CREATE OR REPLACE FUNCTION recalculer_note_moyenne()
RETURNS TRIGGER AS $$
DECLARE
    moyenne FLOAT;
BEGIN
    SELECT AVG(note) INTO moyenne
    FROM note_ressource
    WHERE id_ressource = COALESCE(NEW.id_ressource, OLD.id_ressource);

    IF moyenne IS NULL THEN
        moyenne = 0;
    END IF;

    UPDATE ressource
    SET note_moyenne = moyenne
    WHERE id = COALESCE(NEW.id_ressource, OLD.id_ressource);

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trigger_recalculer_note_moyenne_insert
AFTER INSERT ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();

```

```

CREATE TRIGGER trigger_recalculer_note_moyenne_update
AFTER UPDATE ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();

```

```

CREATE TRIGGER trigger_recalculer_note_moyenne_delete
AFTER DELETE ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();

```

```

---
-- Trigger pour vérifier la disponibilité d'une salle avant réservation
CREATE OR REPLACE FUNCTION vérifier_disponibilité_salle()
RETURNS TRIGGER AS $$
DECLARE
    conflit BOOLEAN;
BEGIN
    -- Vérifier si la salle est déjà réservée pour cette plage horaire
    SELECT EXISTS (
        SELECT 1
        FROM reservation_salle rs
        WHERE rs.salle_id = NEW.salle_id
        AND NEW.date_debut < rs.date_fin
        AND NEW.date_fin > rs.date_debut
        AND rs.statut = 'confirmée'
    ) INTO conflit;

```

```

IF conflit THEN
    RAISE EXCEPTION 'La salle % est déjà occupée pour cette plage horaire.', NEW.salle_id;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_vérifier_disponibilité_salle
BEFORE INSERT OR UPDATE ON réservation_salle
FOR EACH ROW
EXECUTE FUNCTION vérifier_disponibilité_salle();

---
-- =====
-- 7. VUE SQL POUR LE TABLEAU DE BORD
-- =====
CREATE OR REPLACE VIEW vue_tableau_de_bord AS
SELECT
    u.id AS utilisateur_id,
    u.nom,
    u.prénom,
    u.email,
    u.rôle,
    -- Tâches
    COUNT(CASE WHEN t.statut = 'à_faire' THEN 1 END) AS tâches_à_faire,
    COUNT(CASE WHEN t.statut = 'en_cours' THEN 1 END) AS tâches_en_cours,
    COUNT(CASE WHEN t.statut = 'terminée' THEN 1 END) AS tâches_terminées,
    -- Notifications
    COUNT(CASE WHEN n.statut = 'non_lu' THEN 1 END) AS notifications_non_lues,
    -- Progression
    COALESCE(AVG(p.note), 0) AS note_moyenne,
    -- Échéances
    MIN(e.date_limite) AS prochaine_échéance,
    -- Calendrier
    COUNT(DISTINCT ev.id) AS événements_à_venir,
    -- Groupes
    COUNT(DISTINCT ug.groupe_id) AS groupes_rejoints,
    -- Ressources
    COUNT(DISTINCT r.id) AS ressources_créées,
    -- Séances
    COUNT(DISTINCT s.id) AS séances_à_venir
FROM
    utilisateur u
LEFT JOIN tâche t ON u.id = t.utilisateur_id
LEFT JOIN notification n ON u.id = n.destinataire_id AND n.statut = 'non_lu'
LEFT JOIN progression p ON u.id = p.id_utilisateur
LEFT JOIN échéance e ON u.id = (SELECT t2.utilisateur_id FROM tâche t2 WHERE t2.id = e.id_tâche)
LEFT JOIN événement ev ON u.id = ev.utilisateur_id AND ev.date_debut > NOW()
LEFT JOIN utilisateur_groupe ug ON u.id = ug.utilisateur_id
LEFT JOIN ressource r ON u.id = r.utilisateur_id
LEFT JOIN séance s ON u.id IN (
    SELECT ps.utilisateur_id FROM participation_séance ps

```



```

    JOIN séance s2 ON ps.séance_id = s2.id
    WHERE s2.date_debut > NOW()
)
WHERE
    u.statut = 'actif'
GROUP BY
    u.id, u.nom, u.prénom, u.email, u.rôle;

---
-- =====
-- 8. RÈGLES DE SÉCURITÉ (RLS)
-- =====
-- Activer RLS sur toutes les tables
ALTER TABLE utilisateur ENABLE ROW LEVEL SECURITY;
ALTER TABLE etablissement ENABLE ROW LEVEL SECURITY;
ALTER TABLE cursus ENABLE ROW LEVEL SECURITY;
ALTER TABLE cours ENABLE ROW LEVEL SECURITY;
ALTER TABLE utilisateur_cours ENABLE ROW LEVEL SECURITY;
ALTER TABLE ressource ENABLE ROW LEVEL SECURITY;
ALTER TABLE tâche ENABLE ROW LEVEL SECURITY;
ALTER TABLE échéance ENABLE ROW LEVEL SECURITY;
ALTER TABLE événement ENABLE ROW LEVEL SECURITY;
ALTER TABLE calendrier ENABLE ROW LEVEL SECURITY;
ALTER TABLE groupe ENABLE ROW LEVEL SECURITY;
ALTER TABLE utilisateur_groupe ENABLE ROW LEVEL SECURITY;
ALTER TABLE notification ENABLE ROW LEVEL SECURITY;
ALTER TABLE demande_reset_mot_de_passe ENABLE ROW LEVEL SECURITY;
ALTER TABLE salle ENABLE ROW LEVEL SECURITY;
ALTER TABLE réservation_salle ENABLE ROW LEVEL SECURITY;
ALTER TABLE séance ENABLE ROW LEVEL SECURITY;
ALTER TABLE participation_séance ENABLE ROW LEVEL SECURITY;
ALTER TABLE note ENABLE ROW LEVEL SECURITY;
ALTER TABLE commentaire_note ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE question_quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE reponse_quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE progression ENABLE ROW LEVEL SECURITY;
ALTER TABLE donnees_analytiques ENABLE ROW LEVEL SECURITY;
ALTER TABLE note_ressource ENABLE ROW LEVEL SECURITY;

-- Politiques RLS pour Utilisateur
CREATE POLICY "Les utilisateurs peuvent gérer leur propre profil"
ON utilisateur
FOR ALL
USING (auth.uid() = id::TEXT);

-- Politiques RLS pour Tâche
CREATE POLICY "Les utilisateurs peuvent voir et gérer leurs tâches ou celles de leur groupe"
ON tâche
FOR ALL
USING (
    utilisateur_id = auth.uid()::UUID
    OR groupe_id IN (
        SELECT groupe_id FROM utilisateur_groupe

```

```

        WHERE utilisateur_id = auth.uid()::UUID
    )
);

-- Politiques RLS pour Groupe
CREATE POLICY "Les utilisateurs peuvent voir les groupes dont ils sont membres"
ON groupe
FOR SELECT
USING (
    créateur_id = auth.uid()::UUID
    OR EXISTS (
        SELECT 1 FROM utilisateur_groupe
        WHERE groupe_id = groupe.id
        AND utilisateur_id = auth.uid()::UUID
    )
);

-- Politiques RLS pour Utilisateur_Groupe
CREATE POLICY "Les utilisateurs peuvent gérer leur appartenance aux groupes"
ON utilisateur_groupe
FOR ALL
USING (utilisateur_id = auth.uid()::UUID);

-- Politiques RLS pour Séance
CREATE POLICY "Les utilisateurs peuvent voir les séances de leurs groupes"
ON séance
FOR SELECT
USING (
    groupe_id IN (
        SELECT groupe_id FROM utilisateur_groupe
        WHERE utilisateur_id = auth.uid()::UUID
    )
    OR créateur_id = auth.uid()::UUID
);

-- Politiques RLS pour Participation_Séance
CREATE POLICY "Les utilisateurs peuvent gérer leur participation aux séances"
ON participation_séance
FOR ALL
USING (utilisateur_id = auth.uid()::UUID);

-- Politiques RLS pour Notification
CREATE POLICY "Les utilisateurs peuvent voir leurs propres notifications"
ON notification
FOR ALL
USING (destinataire_id = auth.uid()::UUID);

-- Politiques RLS pour Ressource
CREATE POLICY "Les utilisateurs peuvent voir les ressources de leurs groupes ou cours"
ON ressource
FOR SELECT
USING (
    utilisateur_id = auth.uid()::UUID
    OR groupe_id IN (

```

```

        SELECT groupe_id FROM utilisateur_groupe
        WHERE utilisateur_id = auth.uid()::UUID
    )
    OR id_cours IN (
        SELECT cours_id FROM utilisateur_cours
        WHERE utilisateur_id = auth.uid()::UUID
    )
);

-- Politiques RLS pour Note
CREATE POLICY "Les utilisateurs peuvent voir les notes de leurs groupes"
ON note
FOR SELECT
USING (
    groupe_id IN (
        SELECT groupe_id FROM utilisateur_groupe
        WHERE utilisateur_id = auth.uid()::UUID
    )
);

-- Politiques RLS pour Commentaire_Note
CREATE POLICY "Les utilisateurs peuvent gérer leurs commentaires sur les notes"
ON commentaire_note
FOR ALL
USING (auteur_id = auth.uid()::UUID);

-- Politiques RLS pour Calendrier
CREATE POLICY "Les utilisateurs peuvent gérer leur propre calendrier"
ON calendrier
FOR ALL
USING (id_utilisateur = auth.uid()::UUID);

-- Politiques RLS pour Événement
CREATE POLICY "Les utilisateurs peuvent voir les événements de leur calendrier ou groupe"
ON événement
FOR SELECT
USING (
    utilisateur_id = auth.uid()::UUID
    OR groupe_id IN (
        SELECT groupe_id FROM utilisateur_groupe
        WHERE utilisateur_id = auth.uid()::UUID
    )
    OR calendrier_id IN (
        SELECT id FROM calendrier
        WHERE id_utilisateur = auth.uid()::UUID
    )
);

-- Politiques RLS pour Quiz
CREATE POLICY "Les utilisateurs peuvent voir les quiz de leurs cours"
ON quiz
FOR SELECT
USING (
    id_utilisateur = auth.uid()::UUID

```

```
OR id_cours IN (  
    SELECT cours_id FROM utilisateur_cours  
    WHERE utilisateur_id = auth.uid()::UUID  
)  
);  
  
-- Politiques RLS pour Demande_Reset_MotDePasse  
CREATE POLICY "Les utilisateurs peuvent voir leurs propres demandes de reset"  
ON demande_reset_mot_de_passe  
FOR ALL  
USING (utilisateur_id = auth.uid()::UUID);
```

Explications des Ajouts et Modifications

1. Entités Ajoutées pour les Scénarios

Entité	Rôle	Lien avec les scénarios
Demande_Reset_MotDePasse	Gère les demandes de réinitialisation de mot de passe.	Scénario 1 : Pierre demande un nouveau mot de passe après 3 échecs.
Salle	Stocke les informations sur les salles (ex: "Je bosse").	Scénario 2 : Paul réserve une salle pour ses séances.
Réservation_Salle	Lie une salle à une séance et vérifie les conflits.	Scénario 2 : Vérification que "Je bosse" est occupée pour la 2ème séance.
Séance	Organise les sessions de travail (date, heure, lieu).	Scénario 2 : Paul planifie 2 séances.
Participation_Séance	Suivi des validations/refus des membres pour chaque séance.	Scénario 2 : Marie valide les 2 séances, Jeanne seulement la 1ère.
Note	Permet aux membres d'un groupe d'ajouter des notes collaboratives.	Scénario 3 : Paul ajoute une note dans le groupe.
Commentaire_Note	Permet de commenter les notes.	Scénario 3 : Marie et Jeanne interagissent avec la note de Paul.

2. Champs Ajoutés/Modifiés

Table	Champ	Type	Rôle
Utilisateur	compteur_echecs_connexion	INTEGER	Compte les échecs de connexion (Scénarios 1).
Utilisateur	dernière_tentative_connexion	TIMESTAMP WITH TIME ZONE	Horodate la dernière tentative (Scénarios 1).
Utilisateur	statut	statut_utilisateur (ENUM)	Gère les états du compte (actif/bloqué/suspendu).
Séance	obligatoire	BOOLEAN	Distingue les séances obligatoires des optionnelles (Scénarios 2).
Tâche	calendrier_id	UUID (FK)	Lie une tâche à un calendrier pour synchronisation (Scénarios 3).
Ressource	groupe_id	UUID (FK)	Permet de lier une ressource à un groupe (Scénarios 3).
Événement	séance_id	UUID (FK)	Lie un événement à une séance (pour le calendrier).

3. Triggers Ajoutés

Trigger	Table	Action
trigger_block_account_after_failures	Utilisateur	Bloque un compte après 3 échecs de connexion (Scénarios 1).
trigger_vérifier_disponibilité_salle	Réservation_Salle	Empêche les doubles réservations de salles (Scénarios 2).
trigger_sync_séance_événement	Séance	Crée automatiquement un événement dans le calendrier pour chaque séance (Scénarios 2/3).

<code>trigger_protect_créateur_groupe</code>	<code>Utilisateur_Groupe</code>	Empêche la suppression du créateur d'un groupe.
--	---------------------------------	---

4. Vues et RLS

- **Vue** `vue_tableau_de_bord` : Centralise les données pour le tableau de bord utilisateur (tâches, notifications, séances, etc.).
 - **RLS** : Politiques de sécurité pour limiter l'accès aux données selon le rôle de l'utilisateur (ex: un étudiant ne voit que ses propres tâches ou celles de ses groupes).
-

Points à Valider

1. Noms des tables :

- J'ai renommé `groupe_collaboratif` en `groupe` et `membre_groupe` en `utilisateur_groupe` pour plus de cohérence. Est-ce acceptable ?
- De même, `échéance` est conservée, mais elle pourrait être fusionnée avec `Tâche` si une échéance = une tâche avec une date limite.

2. Champs optionnels :

- Certains champs comme `Note.titre` ou `Tâche.description` sont optionnels. Souhaitez-vous les rendre obligatoires ?

3. Synchronisation du calendrier :

- Les triggers `sync_tâche_événement` et `sync_séance_événement` créent des événements pour **tous les membres d'un groupe**. Cela peut générer beaucoup de données. Une alternative serait de calculer les événements à la volée (via une vue) plutôt que de les stocker.

4. Gestion des conflits de salles :

- Le trigger `vérifier_disponibilité_salle` empêche les doubles réservations. Voulez-vous ajouter une **file d'attente** ou un système de **réservation prioritaire** ?

5. Intégration avec Supabase Auth :

- Les champs `email` et `mot_de_passe` dans `Utilisateur` peuvent être remplacés par l'authentification native de Supabase (via `auth.users`). Souhaitez-vous adapter le schéma pour utiliser **uniquement** `auth.uid()` comme identifiant ?
-

Prochaines Étapes

1. **Valider le MCD** : Ce modèle répond-il à tous vos besoins pour les scénarios et l'existant ?
 2. **Tester avec des données** : Voulez-vous que je génère un **jeu de données d'exemple** pour tester les requêtes ?
 3. **Optimiser** :
 - Faut-il ajouter des **index supplémentaires** (ex: sur **Séance.obligatoire**) ?
 - Faut-il **fusionner certaines tables** (ex: **Échéance** et **Tâche**) ?
 4. **Intégrer l'IA** : Où souhaitez-vous intégrer **Mistral AI** (ex: suggestions de tâches, résumés automatiques des notes) ?
-

Question : Souhaitez-vous que je génère un **diagramme visuel** (SVG ou Mermaid) de ce MCD pour une meilleure visualisation ? Ou préférez-vous passer directement à l'implémentation ?